

PROJET DEVSECOPS

Harbor + Terraform + Pipeline CI sécurisée

Déploiement d'un registre d'images privé, sécurisé de bout en bout et observable.

Dylan TREPON

Objectif & architecture

Déployer Harbor (registre d'images privé) par Infrastructure as Code, sécuriser sa chaîne de livraison, et la rendre observable.

1

Infrastructure as Code

Terraform · Helm · k3s

2

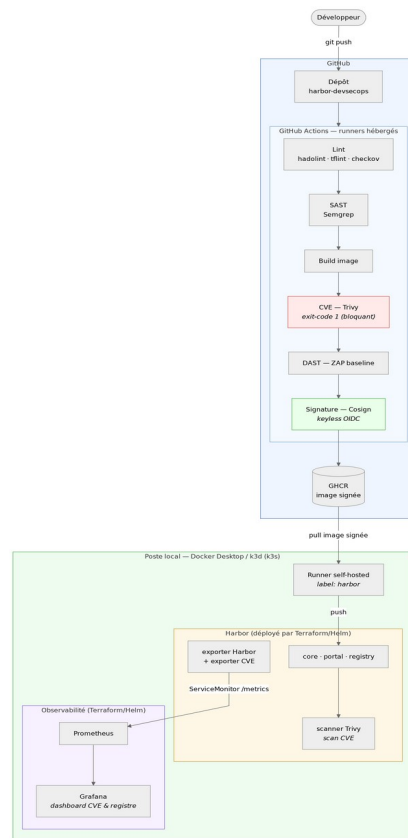
CI sécurisée

lint · SAST · CVE · DAST · signature

3

Observabilité

Prometheus · Grafana



PILIER 1

Infrastructure as Code

k3s / k3d

Le cluster Kubernetes léger (k3s lancé dans Docker via k3d).

Helm

Le gestionnaire de paquets : installe les applications via des charts.

Terraform

L'orchestrateur : pilote Helm pour déployer Harbor et la supervision.

Déploiement

```
terraform init  
terraform plan  
terraform apply
```

Infrastructure reproductible, versionnée et idempotente.

Pipeline CI sécurisée

1

Lint

Dockerfile + IaC

2

SAST

Semgrep (code)

3

CVE

Trivy (bloquant)

4

DAST

ZAP (appli)

5

Signature

Cosign

Bloquante

Un contrôle qui échoue (ex. Trivy, exit-code 1 sur CVE corrigé) arrête la pipeline : l'image n'est pas publiée.

Publication interne

L'image signée part sur GHCR, puis un runner self-hosted la pousse dans Harbor (registre local, hors du cloud).

PILIER 3

Observabilité

1. Harbor

expose ses métriques + scanne (Trivy)

2. ServiceMonitor

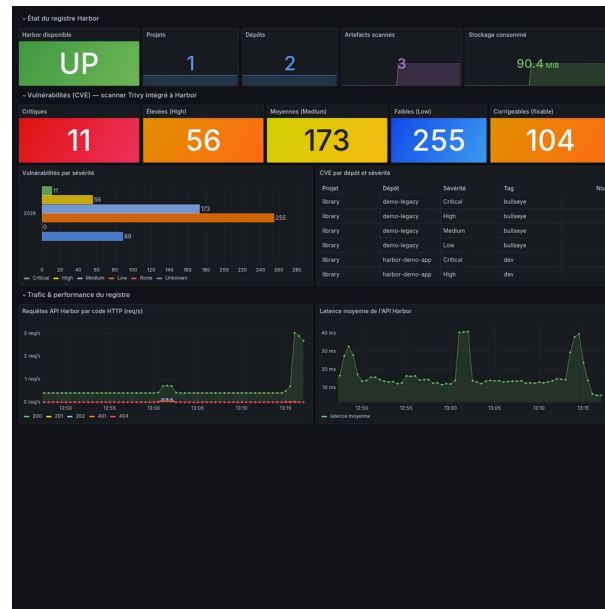
indique à Prometheus de collecter (CRD)

3. Prometheus

collecte et stocke les métriques

4. Grafana

affiche le dashboard Register & CVE



Le dashboard suit les vulnérabilités (CVE) par sévérité, pas seulement la santé du registre.

Résultats

Infrastructure

Harbor + supervision déployés par Terraform sur k3s

Sécurité

Pipeline CI bloquante : lint, SAST, CVE, DAST, signature

Observabilité

Dashboard Grafana des vulnérabilités (CVE) du registre

github.com/KaTaKuRi-31/harbor-devsecops

Merci — démonstration possible : exécution de la pipeline, Harbor et Grafana en direct.